Mentorship Proposal

In this brief document, I would like to outline three key practical topics I am most interested in for our better understanding of what areas of Mathematics I should primarily focus on. I decided not to include purely Cryptographic topics considering our focus on Pure Mathematics.

Understanding ecpairing. Elliptic Curve bilinear pairing (or *ecpairing* for short) is a cornerstone of many modern cryptography protocols. For instance, a large class of zk-SNARKs (succinct non-interactive argument of knowledge) are based on pairing-friendly elliptic curves [1, 2]. Very roughly, given a cyclic group $\mathbb{G} = \langle g \rangle$ of (typically prime) order q, the user might conceal some value $\alpha \in \mathbb{Z}/q\mathbb{Z}$ by computing the point $g^\alpha \in \mathbb{G}$. Given three "encrypted" points $(u, v, w) = (g^\alpha, g^\beta, g^\gamma)$, one can then check linear properties between α, β, γ without finding them explicitly. For instance, by checking $u^2v^3 = w^5$ we can verify whether $2\alpha + 3\beta = 5\gamma$. However, what if we wanted to check whether $\alpha \cdot \beta = \gamma$? There is no way to do that without any more advanced algebraic structure. For that reason, cryptographers used prior research studies from algebraic geometrists and constructed an *efficient* and *secure* bilinear map $e : \mathbb{G} \times \mathbb{G} \to \mathbb{G}_T$ with $\mathbb{G}$ being a special group of points on elliptic curve*. Such mapping satisfies $e(g^\alpha, g^\beta) = e(g^{\alpha\beta}, g) = e(g, g^{\alpha\beta}) = e(g, g)^{\alpha\beta}$. That said, to check whether $\alpha \cdot \beta = \gamma$ one can check whether $e(u, v) = e(w, g)$.

Now, the construction of Weil and Tate pairing are highly complicated and cryptographers even have simplified 150-page documents just to understand how it is computed [3]. While I have practically implemented pairing during one of my projects, I still have no idea (from the Mathematical perspective) about the nature of this object. So I am highly interested in understanding this and related concepts of elliptic curves. Additionally, I would like to explore what makes such construction secure.

Complex Algebraic Structures for Cryptography. One additional interesting question that arises in Cryptography is as follows:

> Is it possible to construct an **efficient** and **secure** multilinear maps (at the very least, efficient 3-way multilinear map $e(\cdot, \cdot, \cdot)$)?

While algebraic geometry has natural generalizations of pairings on curves, they are not efficiently computable (meaning, we cannot practically implement them). Dan Boneh in his awesome lecture "Pairings in Cryptography" provides an awesome overview of pairings and, in particular, discusses why cryptographers care about more complex multilinear maps.

That said, it is interesting to check which concepts of algebraic geometry might become useful for cryptographic applications. I should note that this is not limited to pairings. For instance, I would be glad to read about Hyperelliptic cryptography [4], where instead of elliptic curves, cryptographers try to come up with more sophisticated curves. In particular, a "Handbook of Elliptic and Hyperelliptic Curve Cryptography" seems to be an interesting book on this topic (besides, I happen to have a physical copy in my office).

Optimizing Elliptic Curve-based Protocols. This is also an exciting topic! One of my commerical projects consisted in optimizing elliptic curve operations (namely, scalar multiplication and ecpairing), implemented as the arithmetical circuit over PlonK-based zero-knowledge system. In layman's terms, the only difference from standard implementation is that in such arithmetical systems, modular inverse costs the same as modular multiplication. This, in turn, resulted in numerous nice mathematical optimizations of these operations. For instance, one paper suggested

*I am actually oversimplifying the explanation since e in practice is *assymetric* bilinear map where two inputs are different elliptic curves, one being the twist of another. But I hope I made the high-level explanation clear enough.

using Algebraic tori for such systems to make implementation faster [5] (using the same technique in the "regular" setup is senseless). Additionally, there are numerous non-standard techniques to optimize operations over certain elliptic curves: for instance, using projective space or efficient curve endomorphisms. There are even dedicated books on this topic: see "Guide to Elliptic Curve Cryptography".

That said, it could be nice to explore and to systematize such studies and even implement them practically.

References

[1] M. Dellepere, P. Mishra, and A. Shirzad, "Garuda and pari: Faster and smaller SNARKs via equifficient polynomial commitments," Cryptology ePrint Archive, Paper 2024/1245, 2024. [Online]. Available: https://eprint.iacr.org/2024/1245

[2] J. Groth, "On the size of pairing-based non-interactive arguments," Cryptology ePrint Archive, Paper 2016/260, 2016. [Online]. Available: https://eprint.iacr.org/2016/260

[3] C. Costello, "Pairing for beginerrs," 2021. [Online]. Available: https://static1.squarespace.com/static/5fdbb09f31d71c1227082339/t/5ff394720493bd28278889c6/1609798774687/PairingsForBeginners.pdf

[4] M. J. Jacobson Jr, A. J. Menezes, and A. Stein, "Hyperelliptic curves and cryptography," *High primes and misdemeanours: lectures in honour of the 60th birthday of Hugh Cowie Williams*, vol. 41, pp. 255–282, 2004.

[5] Y. E. Housni, "Pairings in rank-1 constraint systems," Cryptology ePrint Archive, Paper 2022/1162, 2022. [Online]. Available: https://eprint.iacr.org/2022/1162